

COSTRUTTORI IN JAVA

I costruttori sono metodi che, come tali, si configurano come metodi che possono avere dei parametri (lista di parametri) e una implementazione.

I costruttori sono dei metodi “speciali” perché hanno delle caratteristiche differenti rispetto agli altri metodi che troviamo all’interno di una classe.

COSTRUTTORE

- Una classe può avere più di un costruttore.
- Ha lo stesso nome della classe di appartenenza.
- Non ha alcun valore di ritorno (neanche void).
- Invocato automaticamente nel momento in cui viene istanziato un oggetto.
- Vi è sempre un metodo costruttore associato ad una classe (COSTRUTTORE DI DEFAULT).

INVOCAZIONE DI UN COSTRUTTORE

Il costruttore viene invocato nel momento in cui si istanzia un oggetto mediante l’operatore NEW (operatore di istanziazione).

Il costruttore non può essere invocato in qualsiasi momento, ma solo quando viene istanziato l’oggetto.

Il costruttore, infatti, ha solamente il compito di inizializzare un’istanza (oggetto) di una classe. → COSTRUTTORE CREA L’OGGETTO.

Esempio:

```
public Persona(String nome, String cognome, int eta)
{
    this.nome=nome;
    this.cognome=cognome;
    this.eta=eta;
}
```

Istanziare l’oggetto di tipo Persona

```
Persona p1 = new Persona(“Mario”, “Rossi”, 50);
```

NOTA BENE: **una classe può avere più di un costruttore purché abbiano firma diversa (parametri formali differenti nel numero e/o nel tipo).**

Esempio:

Per l'invocazione dei metodi si ricorre alla **DOT NOTATION** (oggetto.nomeMetodo()).

Per il metodo costruttore NON si può utilizzare la DOT NOTATION.

OPERATORE NEW

L'operatore new concettuale fa tre cose:

- CREA L'OGGETTO ALLOCANDO LA MEMORIA NECESSARIA.
- INVOCA SULL'OGGETTO UN COSTRUTTORE DELLA CLASSE PASSANDOGLI, EVENTUALMENTE, DEI PARAMETRI.
- **RESTITUISCE UN PUNTATORE ALL'OGGETTO CREATO.**

La quantità di memoria necessaria per contenere un oggetto è determinata univocamente dalla dichiarazione della classe, ovvero dal numero e dal tipo di variabili di istanza (attributi).

Dove viene allocato l'oggetto? Nello heap di sistema (memoria dinamica).

COSTRUTTORE DI DEFAULT

Esempio:

```
public class Persona {  
    private String nome;  
    private String cognome;  
    private int eta;  
    public Persona() {  
    //corpo del metodo è vuoto  
    }  
    ....  
}
```

Il costruttore di default può non essere dichiarato esplicitamente e fatto includere dal compilatore in automatico.

```
public class TestPersona {
    public static void main(String[] args) {
        Persona p = new Persona(); // CHIAMATA AL METODO COSTRUTTORE
    }
}
```

COSTRUTTORE SENZA PARAMETRI

Il costruttore senza parametri si ottiene dal costruttore di default inserendo almeno un'istruzione all'interno del corpo del metodo.

Esempio: scrivo il costruttore senza parametri

```
public Persona() {
    System.out.println("Sono il costruttore senza parametri.");
}
```

....

```
public class TestPersona {
    public static void main(String[] args) {
        Persona p = new Persona();
    }
}
```

Altro esempio di costruttore senza parametri:

```
public Persona() {
    setCognome(" ");
    setNome(" ");
    setCitta("Parma");
    setEta(18);
}
```

```

}

public class TestPersona {

    public static void main(String[] args) {

        Persona p = new Persona();

        p.visualizzaDati();

    }

}

```

Output di stampa
 Cognome:
 Nome:
 Città': Parma
 Eta': 18

```

public void visualizzaDati() {

    System.out.println("Cognome: " + getCognome());

    System.out.println("Nome: " + getNome());

    System.out.println("Città': " + getCittà());

    System.out.println("Eta': " + getEta());

}

```

COSTRUTTORE PARAMETRICO/PARAMETRIZZATO

Lo scopo del costruttore parametrico, oltre a quello di istanziare l'oggetto, consiste nell'assegnare un valore agli attributi definendone lo stato interno.

```

1. public Persona(String cognome, String nome, String città, int età)
    {
        this.cognome=cognome;
        this.nome=nome;
        this.città=città;
        this.età=età;
    }

```

//Classe di testing

```
Public class TestPersona{  
    public static void main(String[] args)  
    {  
        Persona p = new Persona("Rossi", "Mario", "Parma", 18);  
        p.visualizzaDati();  
    }  
}
```

```
2. public Persona (String cogn, String nom, String citt, int e)  
    {  
        setCognome(cogn);  
        setNome(nom);  
        setCitta(citt);  
        setEta(e);  
    }
```

TIPI DI DATO ASTRATTO (ABSTRACT DATA TYPE - ADT -)

Tipi di dato NATIVI

- numeri interi (int)
- numeri decimali (float/double)
- caratteri (char)
- stringhe (string)
- vero/falso (bool)

Il tipo di dato è caratterizzato da due aspetti:

- INSIEME DI VALORI.
- INSIEME DI OPERAZIONI che possono essere eseguite su di esso.

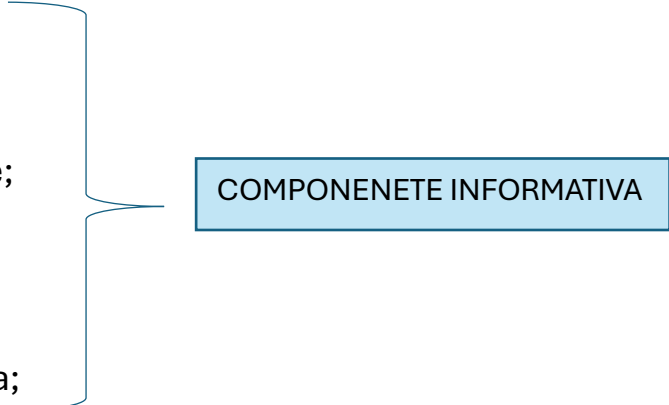
Es. numeri interi (compresi in un intervallo finito); operazioni: somma, sottrazione, divisione, moltiplicazione, elevamento a potenza, ecc.

Tipi di dato NON NATIVI

- STRUCT

```
struct Studente {
```

```
    int matricola;  
    string nome;  
    string cognome;  
    string indirizzo;  
    string citta;  
    int annoNascita;
```



COMPONENETE INFORMATIVA

```
};
```

Quali sono le operazioni che possono essere eseguite sulla struct Studente?

Su questo tipo di dato possono essere eseguite tutte le operazioni (**funzioni/procedure**) che sono state definite dal programmatore.

Un'operazione che può essere eseguita è:

```
int calcolaEta(Studente stud) { ....}
```

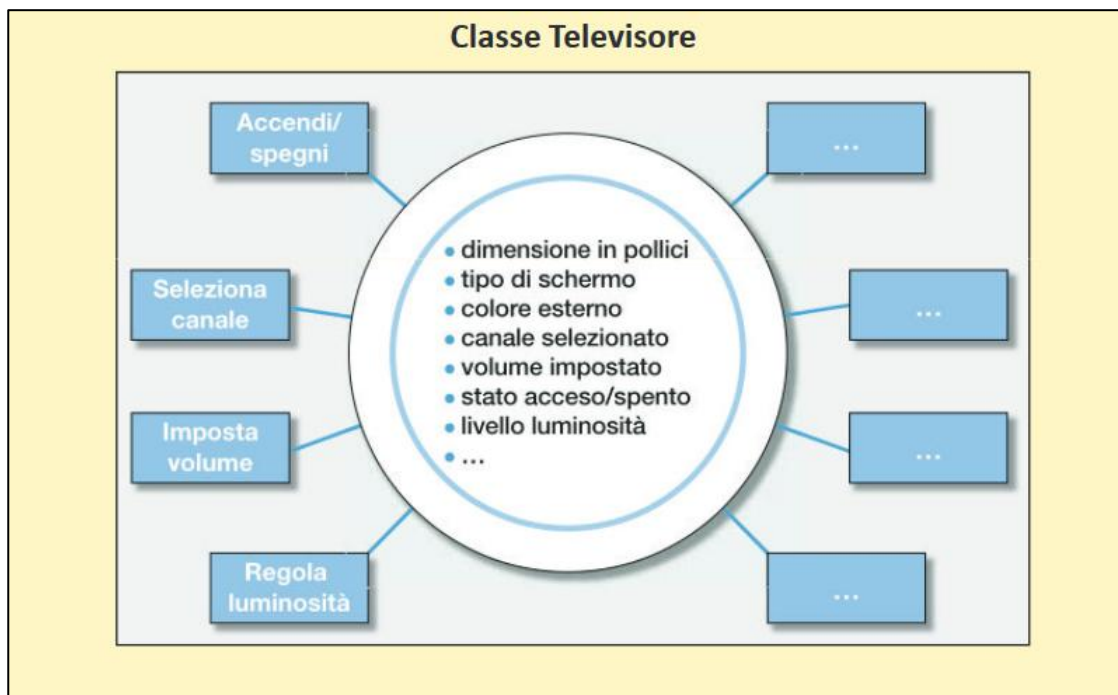
Studente stud → *stud* è una variabile di tipo *Studente*.

Def. **(TIPO DI DATO ASTRATTO)**: tipo di dato formato sia dalla componente informativa (attributi) sia dalle operazioni (metodi) che possono essere implementate sugli oggetti istanziati dalla classe.

La classe, nella programmazione OOP, fa parte dei TIPI DI DATO ASTRATTO: la componente informativa è rappresentata dagli attributi e le operazioni sono rappresentate dai metodi.

Def. **(INTERFACCIA DI UNA CLASSE)**: i metodi di una classe costituiscono la sua interfaccia e costituiscono l'unica modalità per poter interagire con essa (precisazione: con l'oggetto della classe).

- I METODI COSTRUTTORI NON APPARTENGONO ALL'INTERFACCIA DI UNA CLASSE.
- I METODI GETTER E SETTER APPARTENGONO ALL'INTERFACCIA.



Esempio:

```
public class Televisore{
    private int pollici;
    private int luminosita;
    private String TipoSchermo;
    private String colore;
    private int volume;
    private int canale;
    private boolean acceso;

    public Televisore(int pollici, int luminosita, String TipoSchermo,
String colore, int volume, int canale, boolean acceso)
    {
        this.pollici=pollici;
        this.luminosita=luminosita;
        this.TipoSchermo=TipoSchermo;
        this.colore=colore;
        if(volume <0 || volume > 99)
            this.volume=10; //valore di default
        else
            this.volume=volume;
        this.canale=canale;
        this.acceso=acceso;
        //metodi getter/setter
        public int getPollici() {
            return pollici;
        }
        public void setPollici(int pollici) {
            this.pollici = pollici;
        }
    }
}
```



```
public String getSchermo() {  
    return schermo;  
}  
  
public void setSchermo(String schermo) {  
    this.schermo = schermo;  
}  
  
public String getColore() {  
    return colore;  
}  
  
public void setColore(String colore) {  
    this.colore = colore;  
}  
  
public int getCanale() {  
    return canale;  
}  
  
public void setCanale(int canale) {  
    this.canale = canale;  
}  
  
public int getVolume() {  
    return volume;  
}  
  
public void setVolume(int volume) {  
    this.volume = volume;  
}  
  
public int getLuminosita() {  
    return luminosita;  
}  
  
public void setLuminosita(int luminosita) {  
    this.luminosita = luminosita;  
}
```

```

    }

    public boolean getAccesso() {
        return accesso;
    }

    public void setAccesso(boolean accesso) {
        this.accesso = accesso;
    }

    ...    //altri metodi

    ...
}

```

INCAPSULAMENTO

Definizione: indica la proprietà degli oggetti, istanza di una classe, di incorporare al loro interno sia gli attributi sia i metodi.

In pratica, si crea una capsula (barriera concettuale) che isola l'oggetto dall'esterno, in particolare rende non visibili all'esterno della classe gli attributi, ma consente l'interazione con gli oggetti istanziati attraverso i metodi pubblici.

L'incapsulamento è la pratica di nascondere i dettagli interni di un oggetto e di esporre solo ciò che è necessario. Questo è solitamente realizzato con l'uso di **modificatori di accesso** per nascondere gli attributi (rendendoli privati) e fornendo metodi pubblici (getter e setter) per accedere a quegli attributi.

Benefici:

- Protegge lo stato interno di un oggetto.
- Controlla come i dati possono essere modificati o accessibili.
- Riduce la complessità e aumenta la riusabilità del codice.